

FinTech Projesi: Dağıtım ve Kullanım Kılavuzu

Gelişmiş Enflasyon Korumalı ve FIFO Tabanlı Yatırım Takip Platformu

Bu kılavuz, geliştirilen Python/Django tabanlı gelişmiş Fintech uygulamasının yerel (Local) ve üretim (Production) ortamlarında sorunsuz bir şekilde ayağa kaldırılması, bağımlılıkların yönetilmesi ve GitHub mimarisinin kurgulanması için hazırlanmıştır.

1. Güncel Bağımlılıklar (requirements.txt)

Projeye son eklenen `gunicorn` (Production sunucusu), `whitenoise` (DEBUG=False modunda statik dosya yönetimi) ve `openpyxl` (Excel veri işleme motoru) modülleriyle birlikte optimize edilmiş, üretime hazır `requirements.txt` içeriği aşağıdadır:

```
# Temel Geliştirme Framework'ü
Django==6.0.5

# Veritabanı Sürücüsü (PostgreSQL)
psycopg2-binary==2.9.12

# Veri Analizi ve Excel Entegrasyonları
pandas==3.0.3
numpy==2.4.6
openpyxl==3.1.5

# Canlı Finansal Veri ve Kur Servisleri
yfinance==1.3.0
requests==2.34.2
python-dateutil==2.9.0.post0

# Üretim (Production) Ortamı Araçları
gunicorn==26.0.0
whitenoise==6.12.0

# Yardımcı Metin ve SQL Düzenleyiciler
sqlparse==0.5.5
asgiref==3.11.1
```

Bilgi: Paket versiyonları, sisteminizdeki aktif pip list çıktısı ile birebir eşleştirilerek kararlı çalışma garantisi altına alınmıştır. Özellikle Python 3.13 ortamındaki Excel okuma kilitlenmeleri `openpyxl` eklemesi ile çözülmüştür.

2. Kurulum ve Yerel Geliştirme (Local Development)

Projeyi kendi bilgisayarınızda test etmek veya kodlamaya devam etmek için aşağıdaki adımları sırasıyla uygulayınız:

1. Sanal Ortamı Aktif Edin:

```
source .venv/bin/activate
```

2. Yeni Bağımlılıkları Yükleyin:

```
pip install -r requirements.txt
```

3. Veritabanı Göçlerini Tamamlayın ve kullanıcı ekleyin:

```
python manage.py migrate  
python manage.py createsuperuser
```

4. Geliştirme Sunucusunu Başlatın:

```
python manage.py runserver
```

Kritik Uyarı: Eğer yerel ortamda `DEBUG = False` ayarını test etmek istiyorsanız, Django güvenlik amacıyla statik dosyaları (CSS/JS) engeller. Tasarımın kırılmaması için yerel sunucuyu `python manage.py runserver --insecure` komutuyla başlatmalısınız.

3. Üretim Ortamı Dağıtımı (Production Deployment)

Uygulamayı canlı bir Linux (Ubuntu/Debian) sunucusuna taşıırken `DEBUG = False` yapılmalı ve Django asla tek başına (runserver ile) dış dünyaya açılmamalıdır.

Adım 1: Statik Dosyaların Derlenmesi

Canlı ortamda grafiklerin ve Bootstrap simgelerinin (Bootstrap Icons) hatasız çalışması için tüm statik varlıkların tek bir dizinde toplanması gerekir. `settings.py` içinde `STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')` tanımının olduğundan emin olun ve terminalde şu komutu çalıştırın:

```
python manage.py collectstatic --noinput
```

Adım 2: Gunicorn ve WhiteNoise Mekanizması

Canlı sunucuda performans ve kilitlenme (Worker Timeout) yaşamamak için uygulamayı Gunicorn eş zamanlı işçileri (workers) ve arttırılmış zaman aşımı süresiyle başlatmalıyız. Yahoo Finance ve TCMB gibi dış API istekleri gecikebileceğinden zaman aşımı `t = 90` saniye olarak set edilmiştir:

```
gunicorn core.wsgi:application --bind 0.0.0.0:8000 --workers 3 --timeout 90
```

4. Kritik Yönetim Komutları (Management Commands)

Komut	Açıklama / Görev
<code>python manage.py add_inflation</code>	Yereldeki tcmb_enflasyon_verileri.xlsx dosyasındaki tüm geçmiş Yİ-ÜFE verilerini veritabanına topluca enjekte eder.
<code>python manage.py update_inflation_null</code>	Yereldeki Excel dosyasından (tcmb_enflasyon_verileri.xlsx) boş veya eksik ayların Yİ-ÜFE verilerini günceller.
<code>python manage.py update_inflation</code>	TCMB EVDS API'sine bağlanarak canlı veri çekmeye çalışır. Kurumsal IP engeli veya geçersiz API anahtarı durumunda güvenlik gereği işlemi keser.
<code>python manage.py add_forex</code>	Excel tablosundaki resmi TCMB kurlarını sisteme topluca enjekte eder.

5. Finansal Matematik ve FIFO Mimarisi

Uygulama, satılan varlıkların vergiye tabi kârını hesaplarırken Sermaye Piyasası Kurulu (SPK) ve Gelir İdaresi Başkanlığı (GİB) uyumlu enflasyon düzeltmesi yapmaktadır. Satış anında, varlığın alındığı ayın Yİ-ÜFE endeksi ($E_{\{alis\}}$) ile güncel endeks ($E_{\{guncel\}}$) kıyaslanarak düzeltilmiş maliyet bulunur:

$$\text{Düzeltilmiş Maliyet} = \text{Maliyet}_{\{orjinal\}} \times (E_{\{guncel\}} / E_{\{alis\}})$$

Eğer endeks verisi bulunamazsa, sistem çökmez; koruma bloku sayesinde çarpan otomatik olarak 1.00 kabul edilir ve ana para güvenliği korunur. FIFO döngüsündeki her bir adım benzersiz bir mühürleme (hash) algoritması ile veritabanına kaydedilir.